



GaitTwinApp Development Guide

A record of everything built this session, and how to extend it yourself

Digital Health Twin — Multi-Node WBAN Orchestrator

React Native 0.85 · RN CLI · Android

Generated 2026-08-13

Contents

Overview & Architecture

What this app does

The core pattern: Registry → Store → Service → Screen

File map

Everything Built This Session

Documentation (CLAUDE.md)

Bug fixes to the original app

Feature: connection health & auto-reconnect

Feature: live sensor charts

Feature: session/export management

Feature: GPS tracking, live map & weather heat-marks

CSV integrity fixes

Feature: Google Health Connect fitness integration

How to Add a New BLE Sensor Node (worked example: the Posture / chair node)

How to Add a New Health Connect Data Type (worked example: Hydration)

Lessons Learned & Common Pitfalls

Quick-Reference Checklists

1. Overview & Architecture

1.1 What this app does

GaitTwinApp is the mobile half of a thesis project called the "Digital Health Twin": a multi-node wireless body-area-network (WBAN) that streams sensor data from several wearable ESP32 devices (an instrumented gait insole, a smart hydration bottle, and in the future a posture sensor and an environment sensor) directly to a phone over Bluetooth LE. The phone app connects to each device, decodes its binary sensor payload, shows it live, and logs everything to a synchronized CSV file on the phone's Downloads folder — no separate server or laptop required.

Over this session the app grew from a single-screen BLE logger into a small platform: it gained connection-health tracking with auto-reconnect, live trend charts, a session file manager, GPS route tracking with a live map colored by local weather, and an integration with Android's Health Connect that pulls data from any connected fitness band (the motivating case: a Xiaomi Mi Band 10) into its own tab.

1.2 The core pattern: Registry → Store → Service → Screen

Every data source in this app — a BLE node, GPS, or a Health Connect data type — is built the same four-layer way. Once you recognize this pattern, every feature in the app (and every feature you add yourself) reads the same way:

Layer	Responsibility	Example files
Registry	Plain config. No logic, no React, no state. Just "here is the list of things this app knows about" — one entry per BLE node or per Health Connect data type.	<code>NodeRegistry.ts</code> , <code>HealthConnectRegistry.ts</code>
Store	A Zustand store holding the current state for that data source: latest values, connection status, user settings. Nothing in the app talks to hardware directly from the UI — it all goes through a store.	<code>SensorStore.ts</code> , <code>LocationStore.ts</code> , <code>FitStore.ts</code> , <code>ChartStore.ts</code>
Service	The actual engine: connects to BLE, decodes bytes, watches GPS, polls Health Connect, writes CSV files. Runs independently of any screen being open.	<code>BackgroundOrchestrator.ts</code> , <code>LocationService.ts</code> , <code>HealthConnectService.ts</code> , <code>SessionService.ts</code> , <code>FitDataService.ts</code>
Screen / UI	Presentation only. Reads from a store, renders it, and calls service functions on button press. Never touches BLE/GPS/Health Connect APIs directly.	<code>App.tsx</code> , <code>MapScreen.tsx</code> , <code>SessionManagerScreen.tsx</code> , <code>FitDataManagerScreen.tsx</code>

WHY THIS MATTERS FOR YOU

Because the layers are separated this way, adding a new BLE node or a new Health Connect data type never means rewriting the UI. You add one config entry, teach the Service how to decode it, and the existing screens either pick it up automatically (BLE nodes) or need one small loop addition (Health Connect types). Sections 3 and 4 of this guide walk through both, in full.

1.3 File map

Path	What it is
Sensor / BLE pipeline	
<code>src/config/NodeRegistry.ts</code>	List of BLE nodes (GAIT, POSTURE, HYDRATION, ENVIRONMENT) with their service/characteristic UUIDs.
<code>src/store/SensorStore.ts</code>	Live per-node connection status, bindings, latest decoded readings, recording state.
<code>src/services/BackgroundOrchestrator.ts</code>	Scans for BLE devices, connects, decodes payloads, writes the 20ms-tick sensor CSV, handles reconnects.
<code>src/services/SessionService.ts</code>	List/rename/delete recorded sensor CSV sessions.
<code>src/screens/SessionManagerScreen.tsx</code>	UI for the above.
<code>src/store/ChartStore.ts</code> , <code>src/components/Sparkline.tsx</code>	Rolling in-memory history per metric, and the bar-sparkline that renders it.
GPS / Map pipeline	
<code>src/store/LocationStore.ts</code>	Current position, recorded path, weather cache, GPS on/off toggle.
<code>src/services/LocationService.ts</code>	Watches GPS, fetches weather (Open-Meteo, no API key) for the current location.

<code>src/screens/MapScreen.tsx</code> , <code>src/screens/mapHtml.ts</code>	A WebView running a self-contained Leaflet/OpenStreetMap page, fed incrementally from React via <code>injectJavaScript</code> .
Health Connect / Fit pipeline	
<code>src/config/HealthConnectRegistry.ts</code>	List of Health Connect record types this app reads (steps, heart rate, sleep, ...).
<code>src/store/FitStore.ts</code>	Granted permissions, latest value per type, per-type sync watermark, sync settings.
<code>src/services/HealthConnectService.ts</code>	Health Connect init/permissions, the watermark-based sync loop, CSV row builders.
<code>src/services/FitDataService.ts</code> , <code>src/screens/FitDataManagerScreen.tsx</code>	List/rename/delete/preview Fit CSVs.
App shell	
<code>App.tsx</code>	The whole UI: Sensors/Fit tab bar, all data cards, all modals. Presentation-only.

2. Everything Built This Session

2.1 Documentation

The session started by writing `CLAUDE.md` files (one at the thesis-repo root, one inside `server/GaitTwinApp/`) documenting the whole monorepo and this app's architecture in detail, so future work — by you or by an AI assistant — starts from an accurate map instead of re-discovering the codebase from scratch each time.

2.2 Bug fixes to the original app FIXES

An audit of the original code found and fixed nine real bugs:

Bug	Fix
CSV header duplicated if the app restarted mid-session	Header is only written if the file doesn't already exist, checked once at recording-start.
A failed CSV write silently discarded that batch of data	The pending batch is only cleared from memory after the write actually succeeds; on failure it's re-queued.
No detection of a BLE device disconnecting after a successful connection	Real <code>device.onDisconnected()</code> listeners added, feeding a proper 4-state <code>nodeStatus</code> (disconnected/connecting/connected/reconnecting) that drives auto-reconnect.
Recording could start against a node that looked "bound" but wasn't actually connected	A confirm dialog now warns and lets you override, instead of silently logging zeros.
Hydration calibration commands (TARE/CAL_W/CAL_C) always reported "Command Sent" even when the BLE write failed	<code>sendHydrationCommand</code> now returns success/failure and the UI reflects it honestly.
BLE scan errors (Bluetooth off, permission denied) were silently swallowed	Surfaced as visible text in the device-bind modal.
A stale scan-timeout could stop the wrong scan	Timeout is tracked in a ref and cleared properly on every new scan/close/unmount.
Boot only checked one of four requested Android permissions before starting BLE	Now checks the two that actually gate scanning (<code>BLUETOOTH_SCAN</code> + <code>BLUETOOTH_CONNECT</code>) and warns if any are missing.
No validation on calibration number inputs	Rejects empty/non-numeric input before sending.

BONUS FIX

The project's Jest test suite was completely broken before this session (native modules aren't mocked by default). Added `transformIgnorePatterns` to `jest.config.js` and manual mocks under `__mocks__` for every native package used, so `npm test` actually runs again.

2.3 Feature: connection health & auto-reconnect FEATURE

Replaced the old boolean `connectedNodes` array with a real state machine, `nodeStatus: 'disconnected' | 'connecting' | 'connected' | 'reconnecting'`, driven by BLE's own disconnect events. The continuous BLE scan naturally reconnects a dropped node; a 3-second per-node cooldown avoids hammering a device that's actively rejecting connections. The Node Assignments card shows live 4-color status, and a banner warns if a node drops mid-recording.

2.4 Feature: live sensor charts FEATURE

Added `ChartStore.ts` — a small ring buffer (60 samples) per metric, fed at a throttled 4 Hz from the BLE decode callbacks (independent of the 20ms CSV clock). `Sparkline.tsx` renders it as plain colored `View` bars — deliberately no charting library, keeping the app's zero-heavy-dependency style. Every gait and hydration tile grew a small trend sparkline under its number.

2.5 Feature: session/export management FEATURE

Recordings moved into `Downloads/GaitTwin/` with a `.meta.json` sidecar (start time, app version, node bindings) written alongside each CSV. A new Session Manager screen lists, renames, and deletes past recordings. CSV sharing was explicitly scoped out — it needs a `FileProvider` manifest entry, deferred as a separate native-config pass.

2.6 Feature: GPS tracking, live map & weather heat-marks

FEATURE

NATIVE DEPENDENCY

Added `@react-native-community/geolocation` (GPS) and `react-native-webview` (to host a Leaflet map). GPS tracking is tied to the recording session and toggled by a master switch that takes effect immediately, even mid-recording. Local weather (temperature + humidity) is fetched from **Open-Meteo** — a free, keyless API — throttled to once per 1 km moved or 5 minutes, whichever comes first, and used to color-code the route on the map from blue (cold) to red (hot). The map itself is a self-contained HTML/Leaflet page (`mapHtml.ts`) loaded into a WebView, fed new points incrementally via `injectJavaScript` rather than reloading the whole page. Speed comes straight from the GPS fix.

BUG FOUND & FIXED AFTER INITIAL SHIP

The GPS toggle originally only took effect the *next* time "Start Recording" was pressed — flipping it mid-session silently did nothing. Fixed so toggling it while already recording starts or stops tracking immediately. Also added real permission re-requesting (it was only checking, never asking) and visible error banners for permission/GPS failures, since the original version failed completely silently.

2.7 CSV integrity fixes

FIXES

Two more real bugs surfaced once GPS data started flowing into the CSV:

- **Added a `Wall_Clock_ISO` column** (an absolute timestamp, e.g. `2026-07-29T14:32:01.123Z`) right after `System_MS` , so every row has a real-world time, not just a session-relative millisecond counter.
- **Fixed silent column-shift corruption.** The app only ever wrote the CSV header once per filename; every time the row format changed (which happened twice — once for GPS/weather columns, once for the wall-clock column) any file that already existed under that name kept its *old* header while new rows used the *new*, longer format — silently shifting every value one or more columns to the right relative to the header labels. Fixed by comparing the existing file's actual first line against the current header at recording-start; on a mismatch, the app automatically starts a new, timestamp-suffixed file instead of corrupting the old one, and tells you so with an on-screen alert.

The final column order, deliberately, is time and location first, then sensors:

```
System_MS, Wall_Clock_ISO, GPS_Lat, GPS_Lon, GPS_Speed_mps, Weather_Temp_C, Weather_Humidity_Pct,
Gait_Pitch, Gait_Roll, Gait_Yaw, Gait_Heel, Gait_Mid, Gait_Toe, Posture_Spine,
Hydro_Fused_mL, Hydro_Weight_g, Hydro_Cap_mL, Env_Temp, Env_Hum
```

2.8 Feature: Google Health Connect fitness integration

FEATURE

NATIVE DEPENDENCY

You asked for "Google Fit" integration for a Mi Band 10. Google has been shutting down the Google Fit APIs in favor of **Health Connect**, its replacement — so that's what was built instead, via the community library `react-native-health-connect` . Unlike Google Fit, Health Connect needs no Google Cloud project, no OAuth screen, no API key — just a single on-device Android permission dialog, the same pattern as the app's existing BLE/location permissions.

What it does

- A new lightweight **"Fit"** tab next to the existing "Sensors" tab (a plain custom tab bar — no navigation library added).
- Reads 16 record types: steps, distance, heart rate, resting heart rate, sleep sessions, active & total calories, blood oxygen (SpO2), exercise sessions, speed, floors climbed, VO2 max, respiratory rate, body temperature, weight, height. (Reproductive-health record types were deliberately excluded — not relevant to fitness-band tracking.)
- **Auto-syncs every 10 minutes** (5/10/15 min presets in the UI) using a **watermark** per data type — each poll only asks Health Connect for records newer than the last one it successfully saved, so no interval choice can ever create a gap, and an idle poll costs almost nothing.
- Writes to `Downloads/GaitTwin/FitData/Fit_<date>.csv` , one row per record in a single shared "long format" — `Wall_Clock_Start_ISO`, `Wall_Clock_End_ISO`, `DataType`, `Value`, `Unit`, `SourceApp`, `ExtraJSON` — so adding a new record type later never needs a CSV schema change (see Section 4).
- A Fit Data Manager screen (mirrors the Session Manager) lists, renames, deletes, and previews past Fit files with a per-type sparkline.

Native changes this required

<code>android/build.gradle</code>	<code>minSdkVersion</code> raised 24→26 (Health Connect's own hard requirement) — drops Android 7.x support.
<code>AndroidManifest.xml</code>	16 <code>android.permission.health.READ_*</code> lines, one background-data permission, a <code><queries></code> declaration for the Health Connect app, and a required "permissions rationale" activity + its Android-14 alias.
<code>PermissionsRationaleActivity.kt</code>	New file — a minimal screen Health Connect requires apps to have, explaining what data is read and why.

MainActivity.kt

Wires `HealthConnectPermissionDelegate` so permission dialogs work.

HOW THE API DETAILS WERE VERIFIED

Rather than guessing field names, the actual installed package's TypeScript definitions were read directly to confirm exact API shapes — e.g. that Health Connect returns physical quantities as wrapped objects (`distance.inMeters` , `weight.inKilograms` , not plain numbers), and that `readRecords` is paginated via a `pageToken` . A real `./gradlew assembleDebug` and a Metro bundle were both run successfully to catch native and bundling errors before calling the feature done.

3. How to Add a New BLE Sensor Node

Worked example: turning the already-stubbed **Posture node** (your "chair" sensor) from a placeholder into a real, working node. The exact same four steps apply to the **Environment** node, or to any brand-new BLE sensor you build later — only the byte-decoding in Step 3 changes.

CURRENT STATE

`NodeRegistry.ts` already lists `POSTURE` with placeholder UUIDs (`YOUR_POSTURE_SERVICE_UUID`). It already shows up in the app's Node Assignments list and can be "Bound" to a MAC address and toggled on — but `BackgroundOrchestrator.ts` has no connect/decode logic for it yet (look for the comment `// (Future Posture and Environment nodes go here...)`). That's the one real gap these steps close.

1

Fill in the real BLE UUIDs

`src/config/NodeRegistry.ts`

Replace the placeholder strings with the actual service/characteristic UUIDs from your ESP32 firmware's BLE server setup (whatever you set in your Arduino/ESP-IDF code):

```
POSTURE: {
  id: 'POSTURE',
  name: 'Posture-Twin-Node',
  serviceUUID: "your-real-service-uuid-here",
  charUUID: "your-real-characteristic-uuid-here",
},
```

That's the only change needed in this file. Everything downstream (the Node Assignments card, the Bind flow, the active-interest toggle) already works generically off whatever is in this `NODES` object — you don't touch `App.tsx` for this part at all.

2

Check the store already has a field for it (it does, for Posture)

`src/store/SensorStore.ts`

`posture: { spineAngle: number }` and `updatePosture()` already exist — this was left in from the original app. If you were adding a brand-new node instead (not Posture/Environment), you'd add a matching block here first:

```
// 1. Add to the state shape
posture: { spineAngle: number };
updatePosture: (data: Partial<SensorState['posture']>) => void;

// 2. Add to the initial state
posture: { spineAngle: 0 },

// 3. Add the update action
updatePosture: (data) => set((state) => ({ posture: { ...state.posture, ...data } })),
```

Also make sure `nodeStatus` and `nodeBindings` ' initial objects include your node's id — for Posture/Environment they already do (`{ GAIT: ..., POSTURE: ..., HYDRATION: ..., ENVIRONMENT: ... }`).

Add the connect + decode block

`src/services/BackgroundOrchestrator.ts`

This is the real work. Find the comment `// (Future Posture and Environment nodes go here...)` near the bottom of the scan callback, and add a block that mirrors the existing GAIT/HYDRATION ones exactly — same connect-chain shape, same status transitions, same subscription cleanup:

```
// =====
// NODE 3: POSTURE (adjust byte length/offsets to match your firmware)
// =====
if (
  device.id === nodeBindings.POSTURE &&
  activeInterests.includes(NODES.POSTURE.id) &&
  canAttemptConnect(NODES.POSTURE.id, nodeStatus[NODES.POSTURE.id])
) {
  setNodeStatus(NODES.POSTURE.id, 'connecting');

  device.connect()
    .then(dev => dev.requestMTU(128))
    .then(dev => dev.discoverAllServicesAndCharacteristics())
    .then(dev => {
      setNodeStatus(NODES.POSTURE.id, 'connected');

      deviceSubscriptions[NODES.POSTURE.id] = deviceSubscriptions[NODES.POSTURE.id] || {};
      deviceSubscriptions[NODES.POSTURE.id].disconnect = dev.onDisconnected(() => {
        handleNodeDisconnected(NODES.POSTURE.id);
      });

      deviceSubscriptions[NODES.POSTURE.id].monitor = dev.monitorCharacteristicForService(
        NODES.POSTURE.serviceUUID, NODES.POSTURE.charUUID, (err, char) => {
          if (err) {
            handleNodeDisconnected(NODES.POSTURE.id);
            return;
          }
          if (char?.value) {
            const buf = Buffer.from(char.value, 'base64');
            // Match this EXACTLY to your ESP32 struct's byte layout and total size.
            if (buf.length === 4) { // e.g. one 4-byte float
              const spineAngle = buf.readFloatLE(0);
              updatePosture({ spineAngle });
              pushChartSample('posture.spineAngle', spineAngle);
            }
          }
        }
      );
    })
    .catch(() => {
      lastDisconnectTime[NODES.POSTURE.id] = Date.now();
      setNodeStatus(NODES.POSTURE.id, 'disconnected');
    });
}
```

THE ONE THING YOU MUST GET EXACTLY RIGHT

The byte offsets (`buf.readFloatLE(0)`), the expected `buf.length`) must match your ESP32 firmware's struct packing *byte for byte*. If your posture sensor sends, say, a 3-axis accelerometer as three floats, that's a 12-byte payload: `pitch = buf.readFloatLE(0)` , `roll = buf.readFloatLE(4)` , `yaw = buf.readFloatLE(8)` , and your `if (buf.length === 12)` guard. Getting this wrong doesn't crash the app — it just silently produces garbage numbers, so double-check your firmware's struct definition before writing this block.

You also need to destructure your new update function at the top of the scan callback, alongside the existing ones:

```
const { activeInterests, nodeBindings, nodeStatus, setNodeStatus, updateGait, updateHydration, updatePosture } = store;
```

4

Add its columns to the CSV

`src/services/BackgroundOrchestrator.ts`

`Posture_Spine` is actually already in the CSV header (left over from the original app) and already being written from `posture.spineAngle` in the 20ms logger. If you add fields beyond what's already there, you must update **all three** of these together, or the header and the rows go out of sync — exactly the column-shift bug described in Section 2.7:

1. The `CSV_HEADER` string constant — add your new column name(s).
2. The row-building template string inside `startMasterClock()`'s 20ms `setInterval` — add the new value(s) in the *same position*.
3. The hydration offline-burst `historicalRow` template — add the same number of empty placeholder fields (`,,`) in the same position, since that row is built from a different code path and must stay the same column count.

Because the app now detects a header mismatch and automatically forks to a new file (Section 2.7's fix), you can't accidentally corrupt an old file by doing this — worst case, your next recording just starts a new, differently-named CSV and tells you why.

5

(Optional) Give it its own UI card

`App.tsx`

The Node Assignments list, Bind flow, and on/off Switch already work automatically for any node in `NodeRegistry.ts` — nothing to do there. The **Posture Analysis Node** data card already exists in `App.tsx` too, showing `posture.spineAngle`. If you add more fields than just spine angle, add more `gridItem`s to that card the same way the Gait card shows six values, optionally with a `<Sparkline metricKey="posture.spineAngle" />` under each, matching the existing style.

3.1 Same steps for the Environment node

`ENVIRONMENT` works identically: fill in its real UUIDs in Step 1, its `environment: { temp, humidity }` state and `updateEnvironment()` action already exist in the store, its `Env_Temp, Env_Hum` columns already exist in the CSV, and its data card already exists in `App.tsx`. You only need Step 3 — the connect/decode block — matched to whatever your temperature/humidity sensor's payload actually looks like.

4. How to Add a New Health Connect Data Type

This is deliberately much shorter than adding a BLE node, because the Fit pipeline was built specifically to make this cheap. Worked example: adding **Hydration** (Health Connect already tracks water intake logged by many apps).

1

Add one row to the registry

`src/config/HealthConnectRegistry.ts`

```
{ id: 'HYDRATION', recordType: 'Hydration', shape: 'interval',
  permission: 'android.permission.health.READ_HYDRATION',
  unit: 'L', label: 'Hydration', color: '#3182ce' },
```

`shape` must be one of the four the app already knows how to handle — pick whichever matches the new type's structure:

Shape	Use for
<code>instant</code>	A single timestamp + single value (e.g. Weight, SpO2, VO2 Max).
<code>interval</code>	A start/end time span + one summary number (e.g. Steps, Distance, Calories — and Hydration).
<code>session</code>	A start/end span with rich nested detail, no single number (e.g. Sleep, Exercise).
<code>series</code>	A record wrapping many inner timestamped samples (e.g. Heart Rate, Speed).

2

Add the manifest permission line

`android/app/src/main/AndroidManifest.xml`

```
<uses-permission android:name="android.permission.health.READ_HYDRATION" />
```

Add it next to the other 16 `android.permission.health.READ_*` lines. This requires a native rebuild (`./gradlew clean` then reinstall) to take effect, same as any other manifest change.

3

Teach the value extractor how to read it

`src/services/HealthConnectService.ts`

Since `Hydration` is an `interval` shape, add one case to the `extractScalar()` switch statement. The exact field name (`volume` , and its unit accessor) comes from Health Connect's own record shape — check the installed package's type definitions (`node_modules/react-native-health-connect/lib/typescript/types/records.types.d.ts`) rather than guessing, the same way this was verified for the 16 types already in the app:

```
case 'HYDRATION': return { value: record.volume?.inLiters ?? null };
```

That's the only code change needed — the row-builder, CSV writer, watermark sync, permission request, and the Fit tab's tile grid all already loop generically over every entry in `HEALTH_DATA_TYPES` , so your new type appears automatically everywhere once it's granted.

THAT'S IT — THREE SMALL EDITS, ZERO UI CODE, ZERO CSV SCHEMA CHANGE.

This is the payoff of the "long format" CSV decision in Section 2.8: every Health Connect record type, no matter how different its shape, writes into the same seven CSV columns (`Wall_Clock_Start_ISO`, `Wall_Clock_End_ISO`, `DataType`, `Value`, `Unit`, `SourceApp`, `ExtraJSON`).

5. Lessons Learned & Common Pitfalls

- **Any time you add a CSV column, update all three places at once** (header, live-row template, offline-burst-row template) — this was the root cause of two real bugs this session. The app now detects a stale header automatically and forks to a new file rather than silently corrupting data, but it's still better to get it right the first time.
- **BLE byte-decoding mistakes are silent.** A wrong offset or wrong `buf.length` guard doesn't throw an error — it just produces wrong numbers or no update at all. Always double-check against your ESP32 firmware's actual struct packing.
- **Health Connect (and most typed native APIs) wrap physical quantities in unit objects** (`distance.inMeters` , not a plain number) — always check the installed package's real TypeScript definitions before writing extraction code, rather than assuming a shape.
- **New native dependencies need a real rebuild.** Metro's fast refresh only reloads JavaScript — adding a package like `react-native-health-connect` or `react-native-webview` , or changing `minSdkVersion` /the Android manifest, requires `cd android && ./gradlew clean` and a fresh `npm run android` , not just a Metro reload.
- **A toggle that "starts something" should take effect immediately, not just next session.** The GPS-toggle bug (Section 2.6) was exactly this: the switch updated a setting but didn't actually start/stop the underlying watcher unless a new recording session began afterward.
- **Jest doesn't know about native modules by default.** Any new native package (BLE, filesystem, geolocation, WebView, Health Connect) usually needs either a `transformIgnorePatterns` entry (if it ships untranspiled ES modules) or a manual mock under `__mocks__` / (if it touches a native module at import time), or the test suite breaks the moment you import it.

6. Quick-Reference Checklists

Adding a new BLE sensor node

1. `NodeRegistry.ts` — real service/characteristic UUIDs.
2. `SensorStore.ts` — state field + update action (skip if reusing Posture/Environment, already present).
3. `BackgroundOrchestrator.ts` — connect/decode block, matched to your firmware's exact byte layout.
4. `BackgroundOrchestrator.ts` — CSV header + live-row template + offline-burst-row template, all three in sync.
5. `App.tsx` — optional: extra grid items / sparklines on the existing data card if you added fields.

Adding a new Health Connect data type

1. `HealthConnectRegistry.ts` — one config row (id, recordType, shape, permission, unit, label, color).
2. `AndroidManifest.xml` — one `<uses-permission>` line, matching the registry's `permission` field.
3. `HealthConnectService.ts` — one `case` in the relevant extractor function (`extractScalar` , `extractSampleValue` , or `extractSessionSummary` depending on shape), field names verified against the installed package's type definitions.
4. Native rebuild (`./gradlew clean` + reinstall) for the manifest change to take effect.

— End of guide —